AI Compiler Performance Engineering

Auto-Loops Books — ch28

Contents

1 LLM Inference Frameworks and Serving Runtimes **1**

1.1 framework landscape 3
1.2 continuous batching 7
1.3 kv paging serv 11
1.4 framework bottlenecks 14
1.5 compiler integration hooks 17
1.6 Framework decode benchmarking methodology 22
1.7 Key Takeaways 32
1.8 Conclusion 33

Bibliography **35**

Chapter 1

LLM Inference Frameworks and Serving Runtimes

Who owns decode goodput—the inference *framework* that batches requests and manages KV, or the *compiler/runtime* that fuses kernels and cuts bytes/token? The industry default splits responsibility badly: frameworks optimize scheduling and capacity; compilers optimize isolated GEMMs; neither measures the same counters on batch-1 decode (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026). Part VIII opens by naming what frameworks *must* keep (request lifecycle, paging, sampling policy) versus what YiRage and MegaKernel stacks can absorb (intra-step residency, launches/token) (Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). The chapter goal is operational clarity: every engineer can point to the decode step box on a diagram and attach counters (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Chapter 27 flagged codesign and autonomous-kernel trends; Part VIII opens by naming who owns each decode counter when those trends land in production (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025). TaxBreak dense Llama-3.2-1B averages 847.5 kernel launches per output token on H100—often before framework-level batching helps at BS=1 (Vellaisamy *et al.*, 2026). vLLM-style serving reduces KV fragmentation and raises fleet utilization, but leaves operator boundaries inside the decoder step unless a compiler reunifies them (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026; Chen and YiRage Contributors, 2026). Memory-Floor reports 1.259× median speedup on H100 from CUDA Graph replay alone at BS=1, ctx=2048—orchestration relief that paging cannot replace

when bytes/token stay high inside the step (Chen, 2026; NVIDIA, 2024). Those three citations anchor the split: launches, paging, and graphs solve different layers of the same decode story (Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; NVIDIA, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). This chapter maps framework layers to Chapter 1 counters so Part VI–VII compiler theory lands in a deployable stack, not a slide (Chen, 2020; Vellaisamy *et al.*, 2026; SemiAnalysis, 2024; AMD, 2024b,a).

Readers should leave with a deployment taxonomy: classify any production stack into framework-native, partial-export, or compiler-owned decode core (Table 1.1), then profile *both* scheduler queue time and in-step ms/token/bytes/token (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026). Chapter 29 descends into YiRage Layer 5 Persistent Kernel runtime; Chapter 30 stitches the three-party contract (Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Until then, treat every framework feature as a hypothesis that must survive BS=1 counter worksheets (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026). If you remember nothing else, remember the decode step box: everything inside is compiler and runtime turf; everything outside is framework turf (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). Draw that box before every integration meeting and attach Chapter One counters to each side per decode token (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016).

Fregly-style mechanical sympathy applies at the *boundary* between framework and compiler: the framework must not hide allocator behavior that forces HBM spill; the compiler must not assume contiguous KV or static batch shapes that break paging (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022; NVIDIA, 2024; Goodfellow *et al.*, 2016). Part VI–VII taught pass ordering and dialect lowering; Part VIII asks where those passes *attach* in a live server (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016). The answer is never "replace the framework" wholesale—it is to shrink the decode step until launches/token and bytes/token move on the same SKU you ship (Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Dao *et al.*, 2022, 2023; NVIDIA, 2024; Goodfellow *et al.*, 2016).

This chapter also connects backward to Chapter 12 execution modes and Chap-



Figure 1.1: Framework landscape: requests enter scheduling and KV policy; bytes/token and launches/token are decided inside the decode step boundary.

ter 23 LLM compile specialization (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Eager versus graph versus MegaKernel is not only a training-time PyTorch concern—it is reproduced inside serving binaries through framework dispatch, CUDA graphs, and custom fused ops (NVIDIA, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). DeepSeek-scale stacks add expert parallel communication below the scheduler; open components in `deps/DeepEP` and `deps/FlashMLA` show kernels that must compose with paging and batching rather than bypass them (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). Read framework documentation as architecture diagrams with counters attached, not as marketing feature lists (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

1.1 framework landscape

Framework landscape spans HuggingFace *eager* paths, vLLM/SGLang *continuous-batching* servers, TensorRT-LLM/ONNX Runtime *compiled* deployments, and custom C++ runtimes (Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Chen, 2020; Chen and YiRage Contributors, 2026). Each makes different fusion depth assumptions: TRT-LLM may fuse within exported regions; HF PyTorch often does not; vLLM optimizes KV and batching while reusing framework attention backends (Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Chen, 2026; Vellaisamy *et al.*, 2026). SGLang extends the same serving layer with structured generation and radix-cache reuse—still framework-side orchestration unless a compiler owns the decoder cell (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

Production stacks rarely pick a single label from the list above. A common pattern is HuggingFace weights loaded into a vLLM server with FlashAttention

backends and optional TensorRT plugins for narrow subgraphs (Kwon *et al.*, 2023; Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). The compiler engineer job is to diagram where the decode step begins and ends in that stack: from slot assignment and block table lookup through final hidden state before sampling (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Anything inside that window is fair game for fusion and residency optimization; anything outside is framework policy (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). Ambiguity at the boundary is the main reason teams debate whether a regression is serving or compiler (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Compiler engineers should classify any deployment into three buckets (Table 1.1): (1) *framework-native* decode (Python loop + vendor libs); (2) *partial export* (compiled attention or MLP only); (3) *compiler-owned decode core* (YiRage PersistentKernel or MegaKernel inside a thin framework shell) (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Liu *et al.*, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). Bucket choice determines whether your lever is batching policy or bytes/token (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026). Teams that pick bucket (1) for time-to-market but benchmark only fleet tokens/s will misattribute compiler regressions to "scheduling" when launches/token never moved (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025).

HuggingFace vs serving servers. HF `transformers` optimizes researcher ergonomics: dynamic shapes, easy checkpoint swap, and Python-visible tensors (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Production servers trade that flexibility for slot-based scheduling, pinned memory pools, and optional CUDA graphs (Kwon *et al.*, 2023; NVIDIA, 2024; Chen, 2026). Compiler integration rarely starts in HF notebooks—it starts when a serving team exports a decode cell and asks which IR metadata (softmax carriers, page tables) survives the cut (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024).

Research notebooks prove model correctness; serving binaries prove goodput under load (Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Liu *et al.*, 2024; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024b,a;

Table 1.1: Framework deployment buckets and where compiler/runtime leverage applies.

Bucket	**Typical stack**	**Primary goodput lever**
Framework-native	HF eager, vanilla PyTorch loop	Continuous batching, KV paging; launches/token often high (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023)
Partial export	vLLM + compiled attention; TRT-LLM subgraph	Export boundaries; graph capture buckets (NVIDIA, 2024; Davies *et al.*, 2025)
Compiler-owned core	Thin shell + YiRage PK / MegaKernel	Fusion, residency, launch collapse inside step (Chen and YiRage Contributors, 2026; Chen, 2020)

Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Teams should freeze a serving configuration—block size, max batch, dtype—before comparing compiler exports (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Otherwise HF eager baselines become moving targets that invalidate A/B conclusions (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

TensorRT-LLM and compiled paths. TRT-LLM and similar engines push fusion into builder-time regions but still sit behind a framework or C++ server for batching and KV (Davies *et al.*, 2025; NVIDIA, 2024; Goodfellow *et al.*, 2016). Partial export wins when attention or MLP dominates bytes/token; it fails when export boundaries fall *inside* online-softmax state or paged KV gather/scatter (Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; Chen, 2026; Chen and YiRage Contributors, 2026). Profile export candidates on BS=1 decode, not only engine build logs (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen, 2026).

Operational ownership. In production, three teams touch the same decode path: serving (framework), compiler (graph lowering), and hardware (SKU drivers) (Davies *et al.*, 2025; Chen, 2020; Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024b,a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024). Without Table 1.1 vocabulary, incidents become blame loops while bytes/token flatlines

(Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Runbooks should record bucket, shape bucket, paging block size, and export boundary hash alongside fleet dashboards (Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

SGLang and structured serving. Structured generation adds mask and grammar state to the framework layer—still outside compiler fusion unless exported (Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016). RadixAttention-style prefix reuse compounds paging benefits; compiler cores must not invalidate shared prefixes with per-request recompilation (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Multi-hardware delta. Framework support matrices differ: CUDA-first servers dominate cloud; CPU/OpenVINO paths serve edge; Ascend/MACA stacks require vendor runtimes (AMD, 2024a,b; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). YiRage targets the same decode core across backends—framework integration must not reintroduce GPU-only assumptions at the API boundary (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Liu *et al.*, 2024; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024). An integration that hard-codes CUDA graph capture without NPU static-shape buckets will pass GPU CI and fail edge deployment (AMD, 2024a; Chen, 2020; Chen and YiRage Contributors, 2026).

Example 1.1.1. A team runs vLLM for KV paging but keeps PyTorch-eager MLP boundaries: paging wins capacity while launches/token stay TaxBreak-high—framework choice fixed scheduling, not fusion (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2026). Moving to bucket (3) requires exporting the full decode cell with page-table layout IR, not swapping attention alone (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Dao *et al.*, 2022).

Migration path from eager to compiler-owned core. Most teams land in bucket (1), prove fleet demand, then partial-export attention or MLP while paging stabilizes (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026;

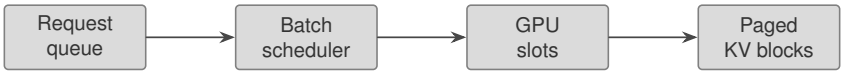


Figure 1.2: Continuous batching architecture: the scheduler interleaves prefill and decode into shared GPU slots backed by paged KV.

Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). The risky middle phase is partial export with hidden boundaries inside RMSNorm or rotary embedding: microbenchmarks show wins while bytes/token flatlines because carriers spill between exported regions (Dao *et al.*, 2022, 2023; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). A disciplined migration keeps the framework scheduler and block allocator fixed while shrinking the decode cell boundary each release, measuring launches/token and bytes/token on the same context buckets after every export change (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016). Skip the worksheet step and you will ship bucket (2) forever, wondering why graph capture helped but cost per million tokens did not (Chen, 2026; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

1.2 continuous batching

Continuous batching interleaves prefill and decode requests to raise GPU occupancy (Kwon *et al.*, 2023; Davies *et al.*, 2025; Liu *et al.*, 2024; Goodfellow *et al.*, 2016). Classic static batching waits until N requests align; continuous batching admits new sequences when others finish prefill or free decode slots—improving *fleet* goodput (tokens/s per GPU) (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026). It does not automatically cut *per-request* bytes/token at BS=1 inside a single decode step (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020). A scheduler that doubles fleet utilization while leaving hundreds of launches per token untouched has solved capacity, not mechanical sympathy inside the decoder (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Serving engineers often first encounter continuous batching as a throughput

win in load tests (Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016). Compiler engineers should insist on paired microbench: same server binary, same paging policy, but forced BS=1 decode cell measurement between scheduler iterations (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). If only fleet metrics move, the next investment belongs in export boundaries and PersistentKernel integration, not longer batching windows (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016).

Prefill vs decode in the scheduler. Prefill is compute-heavy and benefits from large token-parallel batches; decode at BS=1 per sequence is memory- and orchestration-bound (Dao *et al.*, 2022; Davies *et al.*, 2025; Chen, 2026). Frameworks therefore use iteration-level scheduling: each engine step may mix prefill chunks and decode tokens from different clients (Kwon *et al.*, 2023; Davies *et al.*, 2025; Liu *et al.*, 2024). Compiler integration must preserve batching invariants—dynamic sequence lengths, slot assignment, cancellation—while still allowing graph capture or MegaKernel regions for steady decode cells (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Chen, 2026; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016; AMD, 2024b,a; NVIDIA, 2022). Exporting a fused kernel that assumes fixed batch shapes breaks continuous batching unless the runtime provides shape buckets (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020, 2026).

Fleet goodput vs per-request counters. Fleet metrics—tokens/s, cost per million tokens, GPU utilization—reward batching and paging (Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Per-request counters—ms/token, bytes/token, launches/token at BS=1—reward fusion and residency inside the step (Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020). Production teams need both: fleet metrics for capacity planning; per-request counters for compiler A/B and p99 latency under load (Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026). Framework schedulers expose *when* a decode step runs; compilers decide *what* executes inside the step (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Profile both: scheduler queue time vs in-step ms/token (Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020;

Goodfellow *et al.*, 2016).

Document both metric families on the same release dashboard (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Fleet tokens/s may rise while BS=1 bytes/token stalls—label that as a scheduling win, not a fusion win (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Fusion wins that break batching legality should fail integration tests even when microbenchmarks improve (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; NVIDIA, 2024; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). The contract runs both directions: compilers preserve paging and mask invariants; frameworks expose stable plugin ABIs (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Chunked prefill and decode interleaving. Long prefills can starve decode slots; frameworks chunk prefills and interleave decode steps to bound latency (Kwon *et al.*, 2023; Davies *et al.*, 2025; Liu *et al.*, 2024). Compiler-owned cores must support partial prefill shapes without recompilation storms—IR bucket tags for context length and batch composition are framework-facing contracts (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Chen, 2026). DeepSeek-scale MoE adds expert-parallel scheduling on top of token batching (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026).

Scheduling policies and SLOs. Frameworks expose FCFS, priority queues, and preemption for premium tenants (Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016). p99 latency is often scheduler-bound under burst load even when BS=1 kernels are optimal (Davies *et al.*, 2025; Chen, 2026; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Capacity planning should separate queueing SLOs from in-step SLOs (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; SemiAnalysis, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016).

Watermarking and backpressure. When KV pools exhaust, frameworks reject or delay requests (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024b,a; NVIDIA, 2022; Goodfellow *et al.*, 2016). Compiler compression of KV shifts watermarks but requires framework allocator awareness (Liu *et al.*, 2024; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Multi-hardware delta. CPU-served frameworks batch differently—OpenVINO and llama.cpp paths use thread pools and blocked KV rather than GPU slot tables (Goodfellow *et al.*, 2016; AMD, 2024a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020). NPU serving often static-buckets requests by shape; continuous batching semantics must be emulated in software or sacrificed for compile-time legality (AMD, 2024b,a; Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026).

Example 1.2.1. Under load, fleet tokens/s rises 40% after enabling continuous batching, but BS=1 microbench launches/token unchanged—expected if batching fixed occupancy, not fusion (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen, 2026; Chen and YiRage Contributors, 2026). Next lever: compiler-owned decode core inside the same scheduler shell (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024).

Iteration-level scheduling mechanics. Continuous batching implementations maintain a running batch tensor whose rows map to live sequence slots (Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016). Each engine iteration may append new prefill tokens for one client while advancing decode by one token for others in the same launch (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; SemiAnalysis, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016). Compiler-owned cores must accept dynamic active-row masks without recompilation per arrival: shape buckets cover max batch width while runtime masks disable finished sequences (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*,

2016). Framework engineers own the slot table; compiler engineers own legality of fused kernels across masked rows (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016).

1.3 kv paging serv

KV paging serv (PagedAttention and descendants) treats KV cache as allocatable blocks—framework memory management, not compiler math (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022; NVIDIA, 2022, 2024). Without paging, contiguous KV reservations fragment GPU memory as sequences grow and shrink; paging maps logical token positions to physical blocks, improving fleet capacity (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026). Compilers must consume page tables as layout IR (Chapter 13), not fight them with contiguous-KV-only fusion plans (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016).

Paging is the reason large-context serving became economically viable on single-GPU deployments: operators trade small internal fragmentation for predictable allocation (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; NVIDIA, 2022, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016). It is not a substitute for lowering bytes moved during attention itself (Dao *et al.*, 2022, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Teams celebrating paging-only wins should still run Memory-Floor style byte ledgers on decode steps (Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

Block size and attention tiles. Block size trades fragmentation vs attention tile efficiency: smaller blocks reduce waste; larger blocks improve vectorized loads (Kwon *et al.*, 2023; Dao *et al.*, 2022, 2023; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; AMD, 2024b,a; NVIDIA, 2022, 2024). FlashAttention-style kernels assume block-aligned KV access; paging block size should co-tune with compiler tile search on the same context buckets (Dao *et al.*,

2022, 2023; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). A paging policy chosen by SREs without compiler input can force gather/scatter patterns that erase fusion wins (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026).

Run a joint review when either team changes block size or attention tile width (Kwon *et al.*, 2023; Dao *et al.*, 2022, 2023; Davies *et al.*, 2025; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; NVIDIA, 2022, 2024; Goodfellow *et al.*, 2016). The review output is a signed counter sheet, not a slide deck (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Prefix caching and radix reuse. SGLang-style radix caches reuse KV blocks across prompts with shared prefixes—framework-side deduplication (Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Compiler cores must treat cached blocks as valid layout inputs: same page table semantics, different allocation provenance (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025). Exporting attention without page-table metadata breaks both vanilla paging and prefix reuse (Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026; Chen, 2026).

Prefix reuse changes the economics of long system prompts: the framework avoids recomputing shared KV while the compiler still must legalize attention reads against block tables for the suffix tokens (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; SemiAnalysis, 2024; NVIDIA, 2024; AMD, 2024b,a; NVIDIA, 2022; Goodfellow *et al.*, 2016). Compiler teams should test prefix-hit and prefix-miss paths separately because gather patterns and occupancy differ (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Dao *et al.*, 2022; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Treat prefix cache as another paging policy knob co-tuned with tile search (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

MLA and compressed KV. DeepSeek-V3 MLA reduces KV footprint; paging and compiler layout must agree on compressed head dimensions (Liu *et al.*, 2024; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026). Open components under `deps/FlashMLA` illustrate MLA kernel expectations that framework paging must feed (Liu *et al.*,

2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025).

Memory accounting worksheet. Fleet operators should track bytes per KV block, active blocks per sequence, peak concurrent sequences, and fragmentation ratio (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024b,a; NVIDIA, 2022; Goodfellow *et al.*, 2016). Compiler teams add bytes moved per decode step at BS=1 for the same paging policy (Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Disagreement between allocator metrics and profiler HBM counters signals layout IR mismatch (Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Dao *et al.*, 2023; Goodfellow *et al.*, 2016).

Copy and migration costs. Disaggregated prefill/decode copies KV blocks—framework orchestration with compiler-visible layout (Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022, 2023; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024b,a; NVIDIA, 2022; Goodfellow *et al.*, 2016). Fusion plans that assume KV stays local break under migration unless gathers are legal across NUMA or NVLink paths (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Goodfellow *et al.*, 2016).

Multi-hardware delta. GPU paging uses CUDA virtual memory and block pools; CPU paths use pinned host buffers; NPUs use statically reserved SRAM windows (Goodfellow *et al.*, 2016; AMD, 2024a,b; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022, 2024; Dao *et al.*, 2023). A compiler that lowers paged KV only for contiguous GPU tensors will fail on XDNA static layouts (AMD, 2024b,a; Chen, 2020; Chen and YiRage Contributors, 2026).

Example 1.3.1. Block size 16 cuts fragmentation but increases attention loop trips; co-tuning with FlashDecoding tile width recovered bytes/token while keeping paging benefits (Kwon *et al.*, 2023; Dao *et al.*, 2023; Chen, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

PagedAttention internals for compiler authors. Logical token position maps through a block table to physical GPU memory blocks; attention kernels

index KV via block id and offset rather than a flat stride (Kwon *et al.*, 2023; Dao *et al.*, 2022, 2023; Davies *et al.*, 2025; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; SemiAnalysis, 2024; NVIDIA, 2022, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016). Compiler layout IR must represent gathers from block tables as first-class ops with known alignment and dtype, not as opaque custom calls that break fusion legality analysis (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024a,b; NVIDIA, 2022, 2024; Goodfellow *et al.*, 2016). When MegaKernel fusion plans assume contiguous KV tensors, paging forces either illegal fusion or a gather prologue that dominates bytes/token (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Liu *et al.*, 2024; AMD, 2024b,a; NVIDIA, 2022, 2024; Goodfellow *et al.*, 2016). Treat block tables like weight layouts in Chapter 13: co-design block policy with tile search (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

1.4 framework bottlenecks

Framework bottlenecks mirror Chapter 1: launches/token, bytes/token, and orchestration tax—often introduced *above* the decoder kernel by Python loops, dispatcher churn, and unfused vendor op sequences (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; AMD, 2024b,a; NVIDIA, 2022). Framework A/B that reports only tokens/s without per-step byte ledgers misattributes compiler regressions to scheduling (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

When triaging a slow server, split the timeline before optimizing kernels (Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Queue wait implicates admission control and batching policy (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD,

2024a,b; Goodfellow *et al.*, 2016). High in-step milliseconds with moderate launches implicates bytes/token and fusion gaps (Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). High launches with moderate bytes implicates orchestration and graph capture opportunities (Chen, 2026; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Skipping this split sends compiler teams to rewrite attention while the framework still dispatches hundreds of tiny kernels (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Python and host overhead. Eager HF loops pay GIL, tensor metadata, and per-op Python dispatch (Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026). Serving frameworks move hot paths to C++/CUDA but still coordinate sampling, stop criteria, and RPC on host (Kwon *et al.*, 2023; Davies *et al.*, 2025; NVIDIA, 2024; Chen, 2026). CUDA Graph capture removes launch storms for steady decode shapes; it does not fuse illegal HBM traffic (Chen, 2026; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022). Teams that graph-capture unfused graphs lock in high bytes/token at lower launch count (Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; NVIDIA, 2024; Chen and YiRage Contributors, 2026).

Operator chains inside the step. Even "optimized" servers often call separate attention, norm, and MLP kernels each token (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024). TaxBreak counts the launches; Memory-Floor counts the bytes (Vellaisamy *et al.*, 2026; Chen, 2026). Framework bottlenecks end where compiler fusion begins—but only if integration exports the whole decode cell, not a single op (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

Walk one token through the server call stack in a profiler and count CUDA API entries between scheduler entry and sampling exit (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Each entry is a fusion boundary candidate (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Prioritize boundaries that move activations to

HBM between norm and matmul or between attention and MLP (Dao *et al.*, 2022, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; SemiAnalysis, 2024; NVIDIA, 2024; AMD, 2024a,b; NVIDIA, 2022; Goodfellow *et al.*, 2016). That ordered list becomes the export roadmap for bucket (3) migration (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016).

Speculative decoding overhead. Draft models add scheduler complexity: verify steps, acceptance sampling, and dual KV streams (Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016). Frameworks must slot draft and target sequences; compilers should fuse verify cells where numerics allow (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; NVIDIA, 2024). Profile speculative paths separately—fleet speedup can hide verify-step launch storms (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016).

Observability split. Instrument four spans per request: queue wait, prefill steps, decode steps, and sampling/host (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). nsys rows alone are insufficient if framework queue time dominates p99 (Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Export traces with bucket id, block size, and export hash for compiler A/B (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Goodfellow *et al.*, 2016).

RPC and tokenizer tax. Front-end frameworks pay tokenization, detokenization, and gRPC serialization—outside GPU counters but inside user-perceived latency (Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Compiler wins on bytes/token do not remove host bottlenecks; they prevent misallocated engineering weeks (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Multi-hardware delta. CPU-served frameworks hit GIL and memcpy costs; GPU frameworks hit launch storms; NPU frameworks hit static-shape bucket misses (Goodfellow *et al.*, 2016; AMD, 2024a,b; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022, 2024; Dao *et al.*, 2023). Bottleneck triage starts with counter worksheets per target, not framework brand (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Example 1.4.1. Replacing Python loop with CUDA graphs improved ms/token 12% at BS=1 but bytes/token flat—graphs fixed orchestration, not fusion (Chen, 2026; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2020; Dao *et al.*, 2022; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Fair framework comparison methodology. Benchmark posts that compare vLLM, SGLang, and TRT-LLM using only aggregate tokens/s at fixed concurrency often reflect scheduler defaults and paging block sizes rather than intrinsic decode efficiency (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; SemiAnalysis, 2024; AMD, 2024b,a; NVIDIA, 2022; Goodfellow *et al.*, 2016). A fair comparison fixes model checkpoint, dtype, context distribution, block size, and GPU SKU, then reports BS=1 launches/token and bytes/token alongside fleet curves (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Without the fixed-cell worksheet, framework marketing numbers cannot inform compiler roadmap prioritization (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Chapter 1 counters exist precisely to prevent apples-to-oranges serving benchmarks from driving fusion decisions (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

1.5 compiler integration hooks

Compiler integration hooks are the seams where YiRage attaches: exported FX/ONNX regions, custom op *plugin* registration, CUDA graph capture buckets,

and PersistentKernel call sites (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; AMD, 2024b,a; NVIDIA, 2022). Hooks should preserve numerics carriers from Chapter 10 and residency metadata from Chapter 6—exporting a subgraph without those invariants recreates silent HBM spill (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen, 2020; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022).

Integration design meetings often fail because participants use the word kernel loosely (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). A framework custom op may wrap one CUDA kernel or an entire PersistentKernel program with internal loops (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). The hook contract must specify which side owns slot tables, which side owns softmax state, and which side owns graph capture replay (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; NVIDIA, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Ambiguous ownership produces double-free style bugs in performance: two layers both "optimize" the same boundary and break numerics (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

FX and ONNX export regions. PyTorch FX traces decode cells for compiler lowering; ONNX export targets TRT and ORT runtimes (Davies *et al.*, 2025; Chen, 2020; Goodfellow *et al.*, 2016; NVIDIA, 2024; Chen and YiRage Contributors, 2026). Export boundaries must include page-table gathers, rotary embeddings, and online-softmax state—not only matmuls (Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2026; Vellaisamy *et al.*, 2026; Chen, 2020; Davies *et al.*, 2025; Liu *et al.*, 2024; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016). Broken exports pass unit tests on square tensors and fail on paged decode (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022).

Custom op plugins. Frameworks register `torch.library` or vLLM custom

ops that delegate to compiled .so modules (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; NVIDIA, 2022; Dao *et al.*, 2023). Plugin APIs are the stable contract: framework passes slot ids, page tables, and sampling flags; compiler returns fused launch descriptors (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; NVIDIA, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Version skew between framework paging and plugin layout IR is a common production outage mode (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026).

Treat each plugin like a narrow ABI: document struct layouts, alignment, dtype, and error codes; reject silent fallbacks to eager ops (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Fallbacks hide fusion failures behind correct-but-slow paths that pass functional tests while burning fleet budget (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Integration CI should fail when fallback triggers except in explicitly whitelisted debug modes (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

CUDA graph buckets and PersistentKernel. Graph capture requires static launch templates per shape bucket (NVIDIA, 2024; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022). YiRage PersistentKernel mode collapses launches inside the compiled core while the framework keeps scheduling—Chapter 29 details runtime handoff (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022). Integration tests should cover: bucket switch, cancel mid-sequence, and prefix-cache hit (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

Shape buckets should align with framework batch width limits and max context

policies so capture replay does not miss rare production shapes (NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). PersistentKernel programs extend buckets with internal iteration over layers while presenting one host launch per decode step—the pattern Chapter 29 documents in `deps/YiRage` (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Frameworks must not assume one graph per model; they should register one graph per bucket per compiled fingerprint (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Design rule: frameworks own *request and KV lifecycle*; compilers own *decode-step residency*; runtime owns *launch and synchronization* inside the compiled core (Chapter 29) (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022). Chapter 30 formalizes the split for production stacks (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). DeepSeek open components (`deps/DeepEP`, `deps/FlashMLA`) show what must plug below framework schedulers for MoE at scale (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016).

Integration test matrix. Minimum CI scenarios: BS=1 ctx 512/2k/8k; continuous batch mix; prefix-cache hit; cancel mid-stream; bucket switch after compile (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Each scenario logs launches/token and bytes/token (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Rollback and fingerprint caches. YiRage superoptimize caches fused artifacts by IR fingerprint; framework upgrades that change page-table ABI must bump fingerprint salts (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; NVIDIA, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; Goodfellow *et al.*, 2016;

AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Silent cache hits across incompatible paging policies are production footguns (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Multi-hardware delta. Hooks differ by backend: CUDA plugins vs XDNA AIE binaries vs CPU AVX512 kernels—the API shape should stay constant while layout IR lowers per target (AMD, 2024b,a; Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2022, 2024; Dao *et al.*, 2022, 2023; Liu *et al.*, 2024).

Example 1.5.1. vLLM custom op wrapping YiRage superoptimized decode cell: framework passes block tables and slot map; plugin returns single PK launch—launches/token dropped from hundreds to single digits while paging unchanged (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen, 2026; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

ABI stability across releases. Plugin boundaries serialize slot indices, block table pointers, sampling parameters, and optional speculative-draft handles (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Framework minor version bumps that reorder block table fields without coordinated compiler releases produce silent wrong-token bugs worse than a hard crash (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Pin paired framework and compiler versions in production manifests; run golden-token tests on every bump (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen, 2026; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Chapter 30 extends this into a three-party release train (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016).

Metric	Framework layer meaning
Queue wait	Scheduler capacity and admission
Fleet tokens/s	Batching and slot utilization
Block utilization	Paging fragmentation and capacity
Launches/token (in-step)	Orchestration inside decode box
Bytes/token (in-step)	Export boundaries before compiler fusion
Graph bucket hit rate	Capture stability under ctx growth
Plugin fallback rate	Silent eager regressions

Table 1.2: Framework decode counter worksheet (Part VIII).

1.6 Framework decode benchmarking methodology

Benchmarking inference frameworks extends Chapter 1 counters with scheduler-owned and in-step ledgers: queue wait, fleet tokens per second, paging block utilization, launches per token inside the decode box, and bytes per token after drawing the boundary (Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; SemiAnalysis, 2024; AMD, 2024b,a; NVIDIA, 2022; Goodfellow *et al.*, 2016). Build a decode cell with BS=1, ctx buckets $\{512, 2048, 8192\}$, fixed model checkpoint, dtype, paging block size, and GPU SKU—then repeat at representative fleet concurrency for queue wait (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024).

Warm-up must include scheduler steady state, paging allocator warm pools, CUDA graph capture if enabled, and at least one thousand decode steps—single-request microbenches that never cross the decode boundary misattribute framework wins (Kwon *et al.*, 2023; Davies *et al.*, 2025; NVIDIA, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Report queue ms, in-step ms, launches per token, and bytes per token on the same worksheet row (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

Example 1.6.1. Framework gate: at ctx=2048 and BS=1, queue wait must stay within baseline tolerance while in-step launches per token and bytes per token move only with documented plugin or paging changes—attach block size, bucket

id, and decode-box diagram (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024).

Boundary-first worksheets. Draw the decode step box before any framework A/B: slot mask ingress through hidden state before sampling (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Dao *et al.*, 2022; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Counters outside the box are framework-owned; counters inside belong to compiler integration until Chapter 29 runtime handoff (Davies *et al.*, 2025; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Continuous batching benchmarks. Measure BS=1 in-step counters separately from fleet goodput curves (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Batching wins that never appear at batch-1 hide operator-first decode tax (Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Paging policy sweeps. Sweep block size and max-num-seqs against bytes per token and block utilization (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Paging fixes capacity, not activation residency between ops inside a layer (Kwon *et al.*, 2023; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Graph capture versus fusion. Benchmark launches per token alongside bytes per token when enabling CUDA Graphs (NVIDIA, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Graph wins

without byte wins lock in unfused HBM ping-pong (Chen, 2026; NVIDIA, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Plugin integration replay. After framework-only baseline, replay identical traffic through YiRage custom op plugins from Section 1.5 (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Deltas isolate compiler fusion; unchanged queue wait confirms scheduler isolation (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Fair cross-framework comparison. Fix checkpoint, dtype, ctx distribution, block size, and SKU before comparing vLLM, SGLang, or TRT-LLM headline tokens per second (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Publish BS=1 launches per token and bytes per token beside fleet curves (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Speculative decode worksheets. Profile draft and verify paths separately with dual KV stream ledgers (Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Acceptance-rate marketing hides verify-step launch storms (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Observability spans. Instrument queue wait, prefill steps, decode steps, and sampling or host egress per request id (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Export traces with bucket id, block size, and plugin fingerprint for compiler A/B (Davies *et al.*, 2025;

Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Long-context stress. Extend framework benchmarks to ctx=16384 when paging allows; graph buckets tuned at 2K often miss at 32K (Kwon *et al.*, 2023; Dao *et al.*, 2022; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Fail CI when bucket hit rate drops without documented policy change (NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Cancel and prefix-cache stress. Benchmark randomized cancel at 1% rate and prefix-cache hits—framework paths that microbenches skip (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Pool and slot leaks show as rising queue wait with flat in-step counters (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Multi-hardware matrix. Run identical worksheets on GPU, CPU, and NPU serving targets where fleets are heterogeneous (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Framework brand comparisons without fixed-cell worksheets mislead compiler roadmaps (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Regression bisect. When p99 stalls, sequence: (1) split queue wait versus in-step ms; (2) diff launches per token; (3) diff bytes per token; (4) check plugin fallback rate; (5) verify paging ABI semver (Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Skipping boundary split sends compiler teams to rewrite attention while scheduler

queue dominates (Davies *et al.*, 2025; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

CI golden framework pair. Store framework-native and plugin-integrated decode scripts side by side; PRs that raise in-step launches per token or bytes per token without documented paging changes fail merge (Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Golden pairs prevent silent eager relapses at plugin boundaries (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Documentation contract. Publish framework benchmark artifacts with every scheduler or paging promotion: decode-box diagram, block size, ctx buckets, queue wait, in-step counters, and plugin fingerprint (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Without published ledgers, serving teams cannot replay regressions (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Handoff to Chapter 29. Framework worksheets complete when decode-box boundaries and in-step counters are pinned for PersistentKernel integration (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Missing artifacts force runtime teams to guess slot and paging contracts (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Disaggregated prefill-decode benchmarks. When prefill and decode pools split, tag queue wait and in-step counters by pool id (Liu *et al.*, 2024; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Network KV transfer bytes are framework-owned; in-step fusion bytes remain compiler-owned—conflating them hides regressions (Liu *et al.*, 2024; Davies

et al., 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

MoE scheduling benchmarks. Expert-parallel decode adds framework routing tax; count launches per token per layer separately from dense baselines (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Framework wins on fleet tokens per second that ignore routing launches mislead compiler fusion priorities (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Host and RPC tax isolation. Tokenization, detokenization, and gRPC serialization sit outside GPU counters but inside user latency (Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Benchmark host spans separately so compiler byte wins are not credited for framework queue fixes (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Bucket migration cadence. When moving workloads from framework-native to partial-export to compiler-owned decode core per Table 1.1, benchmark after each bucket shrink (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Partial exports that fuse MLP but leave attention eager often show launch wins without byte wins—document both (Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Paging ABI semver gates. Framework minor bumps that reorder block table fields require coordinated plugin releases and golden-token tests (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis,

2024). Benchmark CI must fail when paging ABI semver drifts without fingerprint salt bump (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Efficiency metric. Track *framework-attributed decode efficiency*: fleet tokens per second divided by roofline tokens per second after subtracting in-step bytes and launches explained by compiler artifacts (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Framework teams own queue wait numerator; compiler teams own in-step byte denominator—publish both weekly (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Nightly CI cadence. Run golden framework pairs nightly on pinned GPU runners; plugin replay weekly; multi-hardware legs monthly when pools allow (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Flaky tests that skip paging assertions teach teams to ignore critical framework release gates in every weekly train (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Worksheet archive policy. Version framework worksheets beside paging block size and plugin fingerprints so bisect compares scheduler and in-step counters across releases (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Archive JSON schema fields match on-call postmortem templates for fleet operators (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Executive readout template. One-page promotion summaries show queue wait versus in-step counter deltas side by side—executives approve scheduler changes, not kernel names alone (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao

et al., 2023; NVIDIA, 2022; SemiAnalysis, 2024). Hide in-step regressions behind fleet p50 charts and teams rediscover operator-first serving habits (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Admission control benchmarks. Sweep max concurrent sequences and token budgets; queue wait should move predictably while in-step counters stay flat unless paging policy changes (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Over-admission inflates p99 without touching decode fusion (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Prefix-cache interaction. Prefix hits change prefill mix but must not alter in-step decode counters at BS=1 unless plugin boundaries move (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Benchmark prefix scenarios separately from steady decode cells (Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Quantization and paging. INT8 and FP8 serving paths need explicit scale tensors in framework export regions so plugins do not silently dequantize per op (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Framework benchmarks must pin quant policy beside block size (Liu *et al.*, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Autoscaling warm pools. Cold-start replicas skip warmed graph capture and paging pools—benchmark scale-out events separately from steady decode (NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024;

Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Autoscaling policies must budget warmup time when promotion gates require green worksheets (Chen, 2026; NVIDIA, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Postmortem schema. Store JSON per incident with request id, queue ms, in-step ms, launches per token, bytes per token, block size, bucket id, and plugin fingerprint (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Incidents without schema fields should not close until worksheets are archived (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Final gate sentence. Do not merge framework paging or scheduler changes without green golden pair, paging ABI match, and in-step counter movement on three ctx buckets at BS=1 (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Anything less is costly framework drift wearing compiler fusion labels in production fleets (Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Closing synthesis. Framework decode benchmarking is complete when queue wait, in-step launches per token, and in-step bytes per token align on three ctx buckets with paging and plugin pins—otherwise keep changes in staging (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). That discipline prepares Part VIII runtime and co-design chapters with measurable evidence, not marketing slogans or isolated kernel anecdotes (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Rotary and export boundaries. Framework export regions often stop before rotary embedding and attention gathers—benchmark norm plus QKV plus rotary

as one in-step region (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Skipping rotary in decode-box audits understates framework export tax (Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

SwiGLU export chains. Gate, up, and down MLP projections triple operator boundaries in framework-native stacks (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Framework benchmarks should count in-step launches per token across full MLP chains, not only attention (Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Schedule sympathy at the boundary. Profile queue wait first, paging second, in-step launches third, in-step bytes fourth, then tune framework batch windows (Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Post the boundary sympathy mantra beside every framework worksheet template in the team wiki (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Acceptance criteria recap. Green framework promotion on three ctx buckets requires queue wait flat or improved, in-step launches per token down or flat, in-step bytes per token down, and plugin fallback rate zero except whitelisted debug modes (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Pair with golden-token tests on every paging ABI bump (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Benchmark artifact schema. Store JSON lines per decode step with request id, queue ms, in-step ms, block size, bucket id, launches count, bytes estimate, and ctx length (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Per-step JSON enables fast and reproducible postmortems; aggregates belong in fleet dashboards only after per-step validation passes cleanly (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Onboarding benchmarks. New serving engineers should run decode-box worksheets on day one before proposing batch window or block size changes (Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Onboarding docs must include paired framework-native and plugin-integrated worksheets—culture follows measurable counters at the scheduler and decode boundary every weekly production release train (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

1.7 Key Takeaways

- **Frameworks schedule; compilers fuse.** Continuous batching interleaves prefill and decode requests to raise GPU occupancy and fleet utilization, but it does nothing about the batch-1 bytes/token inside a single decode step (Kwon *et al.*, 2023; Davies *et al.*, 2025). The framework owns request scheduling and the compiler owns intra-step fusion, so a serving win at one layer must not be mistaken for a win at the other (Vellaisamy *et al.*, 2026; Chen, 2026).
- **Paged KV is framework layout the compiler must lower.** PagedAttention treats the KV cache as allocatable blocks—framework memory management, not compiler math—but the compiler must still lower those page tables as first-class IR so fusion can see page boundaries (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026). Hiding the paging from the compiler forces gather/scatter that breaks on-chip residency at the attention seam (Chen, 2020; Davies *et al.*, 2025).

- **Measure both layers, not one.** Framework bottlenecks mirror Chapter 1—launches/token, bytes/token, orchestration tax—but are often introduced *above* the kernel by Python loops and dispatcher churn, so the diagnosis must report queue/scheduling time alongside in-step ms/token and bytes/token (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). A fast kernel behind a slow scheduler still misses the SLO (Chen, 2026; Goodfellow *et al.*, 2016).

- **Integration hooks are contracts, and parity is explicit.** The seams where a compiler attaches—exported FX/ONNX regions, custom-op plugins, CUDA-graph capture buckets, PersistentKernel call sites—must preserve softmax carriers and capture legality across the boundary, or fusion silently regresses (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; NVIDIA, 2024). Framework defaults are CUDA-centric, so multi-hardware parity requires explicit triplet tests on the YiRage backends rather than assumed equivalence (AMD, 2024b,a).

- **Migrate by shrinking the decode export, and version the ABI.** Move a workload bucket by bucket, shrinking the decode export boundary, and never confuse a paging win with a fusion win; before any PersistentKernel work, log the bucket, decode-box boundaries, queue wait, and in-step counters at context 512/2k/8k (Kwon *et al.*, 2023; Davies *et al.*, 2025). Framework paging and compiler plugins must release together with golden-token tests, since an ABI skew between them corrupts output in ways a counter dashboard will not catch (Chen and YiRage Contributors, 2026; Chen, 2026).

1.8 Conclusion

Inference frameworks solve fleet scheduling and KV capacity; they do not replace compiler-side fusion and runtime-side launch collapse (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022). Classify your stack with Table 1.1, profile scheduler and in-step counters separately, and treat every plugin boundary as a numerics and layout contract (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Dao *et al.*, 2022; NVIDIA, 2024; Goodfellow *et al.*, 2016). The practical outcome of this chapter is a shared vocabulary between serving and compiler teams: when paging wins but bytes/token stalls, the next owner is the decode cell export, not the scheduler (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026;

Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; NVIDIA, 2024; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016). Before opening Chapter 29, run one end-to-end worksheet on your production stack: name the bucket, draw the decode step box, log queue wait and in-step counters for ctx 512/2k/8k, and list every plugin boundary with numerics carriers attached (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). That worksheet becomes the acceptance criteria for PersistentKernel integration in the next chapter (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). Chapter 29 descends into YiRage Layer 5 Persistent Kernel runtime (`deps/YiRage` submodule); Chapter 30 stitches framework, compiler, and runtime into one decode goodput workflow (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016).

Bibliography

AMD (2024a). Ai engine technology. `https://www.AMD.com/en/products/adaptive-socs-and-fpgas/technologies/AI-engine.html`. 2, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34

AMD (2024b). AMD XDNA architecture. `https://www.AMD.com/en/technologies/XDNA.html`. 2, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34

Chen, J. (2026). Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode. *arXiv preprint arXiv:2605.30571*. 44-cell cross-GPU study; CUDA Graphs A/B on H100 and L4. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34

Chen, X. (2020). Compiler-based hardware profiling for software-defined architecture. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34

Chen, X. and YiRage Contributors (2026). Yirage: Multi-backend LLM inference optimization. `https://github.com/chenxingqiang/YiRage`. Yield Revolutionary AGile Engine; extends Mirage with CUDA, CPU, MPS, Triton, NKI, and MLIR backends. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34

Dao, T. *et al.* (2023). Flash-decoding for long-context inference. PyTorch blog and Stanford CRFM technical report. Parallelizes attention along KV sequence; up to 8× decode speedup at long context. 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34

Davies, M. *et al.* (2025). Liminal: Exploring the frontiers of LLM decode performance. *arXiv preprint arXiv:2507.14397*. Limit study; DeepSeek-V3 provisions 10× more

decode than prefill nodes. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34

Liu, A. *et al.* (2024). Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34

NVIDIA (2022). Nvidia hopper architecture in-depth. NVIDIA Technical Blog. 5, 6, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34

NVIDIA (2024). Cuda graphs. CUDA Programming Guide. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34

SemiAnalysis (2024). Nvidia tensor core evolution: From volta to blackwell. Newsletter. 2, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34

Vellaisamy, P., Tripathi, S., Natarajan, V., Thenarasu, S. S., Blanton, S., and Shen, J. P. (2026). Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition. *arXiv preprint arXiv:2603.12465*. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34